Chapter 1: Sorting Algorithms

Sorting arranges elements in a defined order. Comparison sorts include merge sort, quicksort and heapsort.

1. Merge sort splits the array in half, sorts each half and merges the results.

2. Quicksort picks a pivot and partitions the array around it.

3. Heapsort builds a max-heap and repeatedly extracts the maximum.

Merge sort runs in T(n) = 2T(n/2) + n = O(n log n) time and is stable.

Lower bound: any comparison sort needs at least log2(n!) ~ n log2 n - 1.44 n comparisons.

Chapter 2: Graph Traversal

A graph G = (V, E) consists of vertices V and edges E.

1. Breadth-first search (BFS) explores neighbours level by level using a queue.

2. Depth-first search (DFS) explores as deep as possible using a stack or recursion.

BFS finds shortest paths in unweighted graphs in O(V + E) time.

Applications: connected components, topological sort, cycle detection.

Chapter 3: Dynamic Programming

Dynamic programming solves problems with optimal substructure and overlapping subproblems.

1. Define the subproblem.

2. Write the recurrence, e.g. F(n) = F(n-1) + F(n-2).

3. Choose memoization or tabulation.

Knapsack: K(i, w) = max(K(i-1, w), v_i + K(i-1, w - w_i)).

Complexity is O(nW) for the 0/1 knapsack problem.

Chapter 4: Summary and Exercises

Summary: choose the algorithm by input size, memory and stability needs.

Exercise 1. Trace merge sort on [5, 2, 9, 1, 7].

Exercise 2. Run BFS from node A on the graph A-B, A-C, B-D.

Exercise 3. Compute F(10) with memoization.